



Databases and its Applications

Relational Modelling

Pooja T S
Computer Applications

Databases and its Applications

Relational Modelling

Cardinality, Participation, Constraints

Pooja T S

Computer Applications



Databases and its Applications

Cardinality, Participation, and Constraints

► Why these concepts matter:

- Define how entities interact in relationships.
- Control the number of occurrences and dependency between entities.
- Form the logical basis for referential integrity and schema translation.



Databases and its Applications

Cardinality — Meaning

- ▶ **Cardinality** specifies the **number of entity instances** that can participate in a relationship.
- ▶ It defines the mapping:
 - **One-to-One (1:1)**
 - **One-to-Many (1:N)**
 - **Many-to-Many (M:N)**
- ▶ Example: One DEPARTMENT **has many** EMPLOYEES.



Databases and its Applications

Cardinality Ratio — Definition

► **Cardinality Ratio** of a relationship set R:

- The maximum number of relationship instances that an entity can participate in.
- Expressed as $1 : 1$, $1 : N$, or $M : N$.

► **Formal Example:**

- If each EMPLOYEE works in one DEPARTMENT, but a department has many employees, then ratio = $1 : N$.



Databases and its Applications

Cardinality Examples — Visual

► One-to-One (1:1)

- Each STUDENT has one ID_CARD.
- Each ID_CARD belongs to one student.

► One-to-Many (1:N)

- One DEPARTMENT employs many EMPLOYEES.

► Many-to-Many (M:N)

- STUDENTS enroll in many COURSES; each course has many students.



Databases and its Applications

Cardinality — ER Notation

► Representation:

- Arrow ($\rightarrow$) from entity to relationship indicates "one".
- Absence of arrow means "many".



Arrow on STUDENT side $\rightarrow$ Each student enrolls in many courses.



Databases and its Applications

Cardinality Ratio in Practice

One-to-One (1:1) Relationship



Explanation: Each PERSON has exactly one PASSPORT, and each PASSPORT belongs to exactly one PERSON. Arrow on both sides $\Rightarrow$ One-to-One.

One-to-Many (1:N) Relationship



Explanation: Each FACULTY teaches many COURSES, but each COURSE is taught by only one FACULTY. Arrow on one side only $\Rightarrow$ One-to-Many.



Databases and its Applications

Cardinality Ratio in Practice

Many-to-One (N:1) Relationship



Explanation: Many STUDENTS enroll in one DEPARTMENT. Arrow on the right side $\Rightarrow$ Many-to-One.

Many-to-Many (M:N) Relationship



Explanation: A STUDENT can register for multiple COURSES, and each COURSE can have multiple STUDENTS. No arrows on either side $\Rightarrow$ Many-to-Many.



Databases and its Applications

Cardinality Ratio in Practice

► Use in database design:

- Helps decide where to place foreign keys.
- Influences normalisation and join strategies.
- Avoids redundancy by modelling correct multiplicities.



Databases and its Applications

Participation — Meaning

- ▶ **Participation** describes whether all entities in a set participate in a relationship.
- ▶ Two types:
 - **Total Participation** — every entity must be related (double line).
 - **Partial Participation** — some entities may not participate (single line).



Databases and its Applications

Participation — Examples

► Example 1 — Total Participation:

- Every EMPLOYEE **must work** in a DEPARTMENT.
- Shown with **double line** between EMPLOYEE and relationship WORKS_IN.

► Example 2 — Partial Participation:

- Not every STUDENT registers in a CLUB.
- Shown with **single line**.



Databases and its Applications

Participation — ER Notation Example



- EMPLOYEE shows **total participation** (double line) in the relationship WORKS_IN.
- DEPARTMENT shows **partial participation** (single line).



Databases and its Applications

Introduction to Relational Constraints

- ▶ **Relational constraints** are rules applied to maintain the **integrity and validity** of data in a relational database.
 - Ensure that data stored across relations remains **accurate, consistent, and meaningful**.
 - Derived from **real-world semantics** captured during ER modelling.
 - Enforced by the DBMS automatically whenever data is inserted, updated, or deleted.
- ▶ Without these constraints, data anomalies such as duplicates, invalid references, or NULL keys can occur.



Databases and its Applications

Why Relational Constraints are Important?

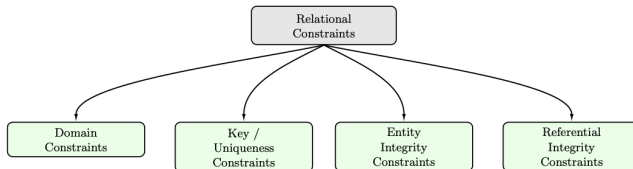
- ▶ They serve as the foundation of **data integrity**.
 - Prevent entry of inconsistent or meaningless data.
 - Maintain logical relationships between tables.
 - Ensure business rules are respected by the database design.
- ▶ Example:
 - A Student must always belong to a valid Department.
 - A Course must have a unique CourseID.
- ▶ Constraints provide an **automatic safeguard** against human error and application-level bugs.



Databases and its Applications

Hierarchy of Relational Constraints

- ▶ Relational constraints are categorised into four fundamental types:



- ▶ Each constraint focuses on a different level of control—**attribute**, **tuple**, or **relationship**.



Databases and its Applications

Domain Constraints

- ▶ **Definition:** Restrict values of attributes to a predefined domain.
 - A domain defines the **data type, range, or permissible set** of values for an attribute.
 - Every attribute value must belong to its domain.
- ▶ **Examples:**
 - Age must be between 0 and 120.
 - Gender { 'M', 'F', 'O' }.
 - Email must follow the pattern `name@domain.com`.
- ▶ **Purpose:** Prevents insertion of invalid or meaningless data.
- ▶ **Note:** Domain constraints are typically implemented as CHECK or DEFAULT conditions in SQL.



Databases and its Applications

Key (Uniqueness) Constraints

- ▶ **Definition:** Ensure that no two tuples in a relation have the same value for a candidate key.
 - A **key attribute** uniquely identifies each tuple.
 - Prevents duplication of primary or candidate key values.
- ▶ **Examples:**
 - In STUDENT(SRN, Name, Email), no two students can have the same SRN.
 - If both SRN and Email are unique, each forms a candidate key.
- ▶ **Purpose:** Maintains entity distinction and data traceability.



Types of Key Constraints — Overview

- ▶ **Key constraints** ensure each record in a relation is **unique and identifiable**.
- ▶ Keys can be classified based on their role and minimality:
 - **Super Key:** Any attribute set that uniquely identifies each tuple.
 - **Candidate Key:** A minimal super key — no subset can uniquely identify tuples.
 - **Primary Key:** The chosen candidate key to act as the main identifier.
 - **Alternate Key:** Candidate keys not chosen as primary.
 - **Composite (Concatenated) Key:** A key made of two or more attributes.
- ▶ Each key enforces a different level of **uniqueness and dependency**.



Types of Key Constraints — Examples

► Examples:

- STUDENT(SRN, Name, Email, Dept)

Super Key: {SRN, Email}, {SRN, Name}

Candidate Keys: {SRN}, {Email}

Primary Key: SRN

Alternate Key: Email

Composite Key: (CourseID, Semester) for ENROLMENT relation

► Additional Types:

- **Foreign Key:** Attribute that references primary key in another table.
- **Partial Key:** Used in weak entities to identify records within owner context.



Databases and its Applications

key Representation

► Notation:

- Primary Key, Dashed underline for Partial Key, Dotted underline for Foreign Key.

STUDENT

DEPENDENT

ENROLMENT



Databases and its Applications

Entity Integrity Constraint

- ▶ **Definition:** Ensures that each tuple in a relation can be uniquely identified.
 - The **primary key cannot be NULL**.
 - Every table must have at least one primary key.
- ▶ **Examples:**
 - In EMPLOYEE(EID, Name, DeptID), EID cannot be NULL.
 - If EID is missing, the record cannot be referenced elsewhere.
- ▶ **Violation:** Inserting a tuple with a NULL primary key breaks entity integrity.
- ▶ **Note:** This rule directly implements the **uniqueness requirement of real-world entities**.



Databases and its Applications

Referential Integrity Constraint

- ▶ **Definition:** Ensures that relationships between tables remain valid.
 - A **foreign key** in one relation must match a **primary key** in another or be NULL.
 - Maintains consistency across related entities.
- ▶ **Example:**
 - EMPLOYEE (DeptID) references DEPARTMENT (DeptID).
 - A new employee record cannot reference a non-existent department.
- ▶ **Purpose:** Protects against orphan or invalid references in multi-table systems.
- ▶ **Hint:** Deleting a parent row with dependents requires either CASCADE or SET NULL.



Databases and its Applications

Comparison of Relational Constraints

► Quick Summary:

Constraint	Ensures	Applied On
Domain	Valid data values	Attribute level
Key	Uniqueness	Tuple / Table level
Entity Integrity	No NULL primary key	Primary key attribute
Referential Integrity	Valid foreign references	Relationship level

- Together, these constraints preserve **data quality, structure, and consistency**.
- Violation of any constraint may lead to anomalies, redundancy, or loss of referential links.



- ▶ Consider a **university database**:
 - **Domain**: Course credits must be 1–5.
 - **Key**: Each course has a unique CourseID.
 - **Entity Integrity**: No course can exist without a valid ID.
 - **Referential Integrity**: A student must enrol only in existing courses.
- ▶ These constraints ensure all relationships (student–course–faculty) remain logically valid.



Databases and its Applications

Participation vs Cardinality

- ▶ **Cardinality** — maximum number of associations.
- ▶ **Participation** — minimum number (whether optional or mandatory).
- ▶ Together define full (min, max) notation.
- ▶ Example:

$(\min, \max) = (1, 3) \Rightarrow$ each entity participates in 1 to 3 relationships.



- ▶ Read relationships in both directions:
 - Department → Employee (1:N) — one dept, many employees.
 - Employee → Department (N:1) — each employee in one dept.
- ▶ Verify minimum participation (must/may).



Databases and its Applications

Common Modelling Mistakes

- Ignoring participation (leads to orphan entities).
- Confusing 1:N and N:1 direction.
- Missing total participation where business rule demands it.
- Treating optional relations as mandatory (adds false constraints).



Databases and its Applications

Case Study — Library ER Constraints

- ▶ Entities: BOOK, MEMBER, ISSUE.
- ▶ Rules:
 - Each book may be issued to **one member at a time**. (1:1)
 - Each member can borrow **many books**. (1:N)
 - Every issue record must link an existing book and member. (Total)



Databases and its Applications

Case Study — Course Registration

- ▶ Entities: STUDENT, COURSE, REGISTRATION.
- ▶ Constraints:
 - Each student can register for several courses. (1:N)
 - Each course can have many students. (M:N)
 - Every registration links one student and one course. (Total)



- ▶ The Entity-Relationship (ER) model converts a real-world problem into a conceptual schema.
- ▶ The goal is to represent data and its relationships **clearly and unambiguously**.
- ▶ Construction follows a logical sequence of steps:
 - Step 1 – Identify Entities
 - Step 2 – Identify Attributes
 - Step 3 – Determine Keys
 - Step 4 – Identify Relationships
 - Step 5 – Specify Cardinality and Participation
 - Step 6 – Identify Weak Entities
 - Step 7 – Refine and Validate
- ▶ Each step refines the previous one until the diagram becomes a complete conceptual view.



Databases and its Applications

Step 1 – Identify Entities

- ▶ Entities are the main objects or concepts in the system.
 - Represent real-world things with independent existence.
 - Each entity becomes a **rectangle** in the ERD.
- ▶ **Examples:**
 - Student, Faculty, Course, Department.
- ▶ Classify entities as:
 - **Strong Entity** – Exists independently.
 - **Weak Entity** – Depends on another entity.
- ▶ **Tip:** Always start by listing all nouns in the problem statement — they often correspond to entities.



Databases and its Applications

Step 2 – Identify Attributes

- ▶ Attributes describe the properties of entities or relationships.
- ▶ Types of attributes:
 - **Simple** – Indivisible (e.g., Name)
 - **Composite** – Can be subdivided ($\text{FullName} = \text{First} + \text{Last}$)
 - **Multi-valued** – Multiple values (PhoneNo)
 - **Derived** – Calculated ($\text{Age} = \text{Today} - \text{DOB}$)
- ▶ Representation:
 - **Ellipse** → Attribute
 - **Double Ellipse** → Multi-valued Attribute
 - **Dashed Ellipse** → Derived Attribute
- ▶ Select meaningful attribute names consistent with data-dictionary conventions.



Databases and its Applications

Step 3 – Determine Keys and Identifiers



- ▶ Each entity set must have a key to uniquely identify its instances.
 - **Primary Key:** Main unique identifier (EmpID)
 - **Candidate Keys:** All possible unique identifiers
 - **Composite Key:** Combination of attributes (CourseID + Semester)
 - **Partial Key:** Identifies weak entity within owner context
- ▶ Representation:
 - Primary Key – single underline
 - Partial Key – dashed underline
 - Foreign Key – dotted underline
- ▶ Example:
 - ▶ STUDENT(SRN, Name, Email) → Primary Key = SRN



Step 4 – Identify Relationships and Participation

- ▶ Determine how entities are associated.
 - **Unary (Recursive)** – Entity relates to itself.
 - **Binary** – Two entities (e.g., Student{Enrolls{Course})
 - **Ternary** – Three entities (e.g.,
Doctor{Prescribes{Medicine{Patient})
- ▶ Representation:
 - **Diamond** → Relationship
 - Connects rectangles (entities) with cardinalities 1:1, 1:N, M:N.
- ▶ Define **participation**:
 - **Total Participation** – All entities participate (double line).
 - **Partial Participation** – Optional participation (single line).



Databases and its Applications

Step 5 – Specify Cardinality

- ▶ Assign correct **cardinality ratios** for each relationship:
 - **1:1** – Each entity relates to one other.
 - **1:N** – One entity relates to many others.
 - **M:N** – Many entities relate to many others.
- ▶ Review for weak entities and dependency relationships.
- ▶ **Validation Checklist:**
 - Every entity has a key.
 - All relationships have correct participation.
 - All attributes are correctly classified.
 - Diagram is readable and logically complete.
- ▶ Once validated, the ERD can be mapped to a relational schema.



PES
UNIVERSITY

CELEBRATING 50 YEARS

Pooja T S
Assistant Professor
Department of Computer Applications
poojats@pes.edu
080-26721983 Extn: 233